

HPS Cholesky: Hierarchical Parallelized Supernodal Cholesky with Adaptive Parameters

SHENGLE LIN, WANGDONG YANG, YIKUN HU, QINYUN CAI, MINLU DAI, HAOTIAN WANG, and KENLI LI, College of Computer Science and Electronic Engineering, Hunan University, China

Sparse supernodal Cholesky on multi-NUMAs is challenging due to the supernode relaxation and load balancing. In this work, we propose a novel approach to improve the performance of sparse Cholesky by combining deep learning with a relaxation parameter and a hierarchical parallelization strategy with NUMA affinity. Specifically, our relaxed supernodal algorithm utilizes a well-trained GCN model to adaptively adjust relaxation parameters based on the sparse matrix's structure, achieving a proper balance between task-level parallelism and dense computational granularity. Additionally, the hierarchical parallelization maps supernodal tasks to the local NUMA parallel queue and updates contribution blocks in pipeline mode. Furthermore, the stream scheduling with NUMA affinity can further enhance the efficiency of memory access during the numerical factorization. The experimental results show that HPS Cholesky can outperform state-of-the-art libraries, such as Eigen LL^T , CHOLMOD, PaStiX and SuiteSparse on 79.78%, 79.60%, 82.09% and 74.47% of 1,128 datasets. It achieves an average speedup of 1.41x over the current optimal relaxation algorithm. Moreover, 70.83% of matrices have surpassed MKL sparse Cholesky on Xeon Gold 6248.

CCS Concepts: • **Mathematics of computing** → **Mathematical software performance**; • **Computing methodologies** → **Symbolic calculus algorithms**; *Algebraic algorithms*; *Parallel algorithms*; Artificial intelligence;

Additional Key Words and Phrases: Supernodal Cholesky factorization, graph convolutional network, hierarchical parallelization, task stream processing, multi-NUMA architecture

ACM Reference format:

Shengle Lin, Wangdong Yang, Yikun Hu, Qinyun Cai, Minlu Dai, Haotian Wang, and Kenli Li. 2024. HPS Cholesky: Hierarchical Parallelized Supernodal Cholesky with Adaptive Parameters. *ACM Trans. Parallel Comput.* 11, 1, Article 3 (March 2024), 22 pages.
<https://doi.org/10.1145/3630051>

The research was partially funded by the National Key R&D Program of China (Grant nos. 2021YFB0300800), the Key Program of National Natural Science Foundation of China (grant no. U21A20461, 92055213, 62227808), Natural Science Foundation of Hunan Province, China (2021JJ50158), and Major Projects of Xiangjiang Laboratory (22xj01011).

Authors' address: S. Lin, W. Yang (Corresponding author), Y. Hu, Q. Cai, M. Dai, H. Wang, and K. Li, College of Computer Science and Electronic Engineering, Hunan University, 116 Lu Shan South Road, Changsha, Hunan, 410082, China; e-mails: {sls036, yangwangdong, yikunhu, hnutsai, daiml, wanghaotian, lkl}@hnu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

2329-4949/2024/03-ART3 \$15.00

<https://doi.org/10.1145/3630051>

1 INTRODUCTION

Solving a large linear system is increasingly intractable in practical scientific and engineering applications, and now it becomes the performance bottleneck in high-performance computation [1, 5, 48]. Especially in the sparse matrix scenario, the performance of the direct solution method is severely affected by the new nonzero items introduced during operation. Hence, how to efficiently solve a large-scale sparse linear system is a critical technique in the field of computational mathematics.

In order to achieve theoretical peak performance of processors, many algorithms have been proposed. Initially, some algorithms such as Envelope [31] and Skyline [4] were designed based on sparse matrix structures to minimize fill-ins. However, due to the irregular memory access, these early algorithms had unsatisfactory performance on multi-core architectures. The *supernodal algorithm* [3, 15, 22, 33], which is based on the elimination tree structure, has shown better computational efficiency in practice. This algorithm consists of two main phases: symbolical analysis and numerical decomposition. It partitions the computational tasks into small dense sub-matrices that can efficiently exploit memory coalescing and BLAS parallelism. Currently, the supernodal algorithm is widely used for solving a sparse **Symmetric Positive Definite (SPD)** system.

In recent decades, the concept of *Relaxed Supernodes* [2] was proposed for merging supernodes that are too small to fully utilize hardware power. *PaStiX* [22] provided a repartitioning and scheduling algorithm for supernodes under memory constraint. Davis and Hager [11] introduced a dynamic supernode strategy and integrated it into the *CHOLMOD* package. However, most existing works [11, 13] choose a fixed threshold to incorporate relaxed supernodes, making the parallel granularity less scalable on modern processors. These approaches result in some remaining nodes that are usually too small to take full advantage of BLAS [16, 17] parallelism. What's more, on the multi-NUMA hardware, the extra overhead of remote memory access still impedes the high performance of numerical factorization. In this context, our main contributions are as follows:

- First, we propose an empirical formula for estimating the total computational overhead of the elimination tree. Based on this, we present a supernodes merging strategy that employs GCN to balance the computational granularity of each task.
- Second, we design a hierarchical parallelization algorithm that efficiently exploits two-level parallelism by coordinating the task nodes of each level with BLAS.
- Third, we describe a streaming task scheduling method that avoids thread context switching between different NUMA and further improves inter-block update efficiency through local memory access.

The experimental results show that our new relaxation parameter empowered by GCN achieves up to 1.33x speedups over traditional methods. In addition, with the optimization of hierarchical parallelism and NUMA affinity, the HPS Cholesky manifests substantial performance gains over *Eigen*, *Cholmod* *SimplicialLL^T*, *Cholmod* *SuperLL^T*, *PaStiX* and *SuiteSparse* with an average speedup of 35.63x, 32.89x, 4.28x, 3.22x and 1.41x on Kunpeng 920 processors, respectively. On Intel Xeon 6248, our approach surpasses *SuiteSparse* and *PaStiX* in 75.18% and 90.22% of the datasets, respectively. Compared with the *MKL sparse Cholesky*, our HPS Cholesky demonstrates great competitiveness on medium-sized sparse matrices with an average speedup ratio of 2.42x.

2 BACKGROUND AND MOTIVATION

2.1 The Elimination Tree and Supernodes

In order to facilitate understanding, we first introduce the block Cholesky method, and then describe the supernodal algorithm. Assume that the first $(j - 1)$ steps of decomposition have been carried out, the block form of Cholesky factorization can be viewed as:

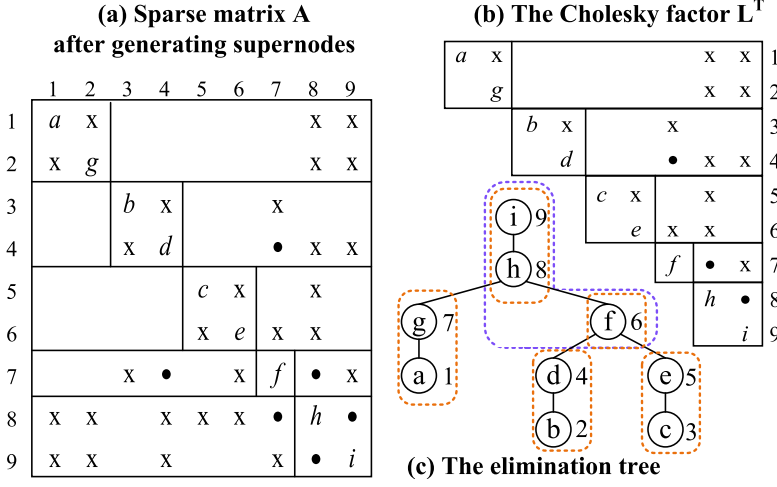


Fig. 1. (a) is an example of an SPD matrix A after reordering. (b) is the Cholesky factor L^T produced by symbolical analysis. The rows with similar structures are merged into subblocks (supernodes) that will be factored in numerical factorization. (c) is the corresponding elimination tree, where the dotted box shows the division of supernodes. Each 'x' denotes a nonzero element and a dot means a fill-in.

$$A = \begin{pmatrix} B & V^T \\ V & C \end{pmatrix} = \begin{pmatrix} L_B & 0 \\ V L_B^{-T} & I \end{pmatrix} \begin{pmatrix} I & 0 \\ 0 & C - V B^{-1} V^T \end{pmatrix} \begin{pmatrix} L_B^T & L_B^{-1} V^T \\ 0 & I \end{pmatrix}, \quad (1)$$

where $B = L_B L_B^T$ is the submatrix Cholesky factorization of the $(j-1)$ -by- $(j-1)$ block. The *Schur complement* $C - V B^{-1} V^T$ represents the remaining part to be factorized, where the suffix part $-V B^{-1} V^T$ is called updating contribution block to the submatrix C .

The key concepts of the supernodal algorithm are the elimination tree and supernodal block, which provide effective management of the inner factorizing and updating process. Figure 1 is an example of (a) a 9×9 SPD matrix A , (b) the factor L^T , and (c) its elimination tree. The symbolical analysis starts by constructing the elimination tree of A , which can obtain the nonzero pattern of each column of factor L . Specifically, the elimination tree is constructed of n nodes corresponding to the columns of matrix A , such that node j is the parent of node i if and only if:

$$node_j = \min\{k > i | L_{i,k}^T \neq 0\}. \quad (2)$$

From the Figure 1(b), node g is the parent of node a since there exists a nonzero element 'X' at $L_{1,2}^T$. The parent of node g is node h because of $L_{2,8}^T \neq 0$.

The *supernodal elimination tree* T is formed based on the fundamental elimination tree. A battery of rows with a similar structure of nonzero elements in factor L^T are merged into supernodes, corresponding to each L_B block in Equation (1). During the numerical factorization, each supernode is factorized first and will contribute updating part $-V B^{-1} V^T = -(V L_B^{-T})(L_B^{-1} V^T)$ to their parent. Specifically, the contribution block at column j for the matrix is defined as the sum of the outer-product:

$$U_j = - \sum_{k \in T[j] - \{j\}} \begin{pmatrix} l_{k,j}^T \\ l_{k,i_1}^T \\ \vdots \\ l_{k,i_r}^T \end{pmatrix} (l_{k,j}^T \ l_{k,i_1}^T \ \cdots \ l_{k,i_r}^T), \quad (3)$$

Table 1. The Time of SuiteSparse Cholesky under Different Relaxation Parameters Settings

Dataset	NNZ/ 10^3	Density	Factorization time (s)		
			max-zero=128	nrelax=16, zrelax=0.1	default strategy
thermal2	4904.2	0.0003%	4.865	4.814	5.194
af_shell3	9046.9	0.0035%	4.799	4.634	3.570
Dubcova3	1891.7	0.0088%	0.463	0.435	0.445
cfid2	1605.7	0.0105%	3.053	3.389	2.978
boneS01	3421.2	0.0211%	2.169	3.630	2.223

where $T[j]$ is the subtree with root j , $\{i_1, \dots, i_r\}$ are the column subscripts of logical nonzeros in $L_{j,*}^T$, which means it contains all contributions from the preceding rows that are descendants of node j . The asymptotic time taken by such k rank-1 update is $O(\sum_{i=1}^k \sum_{k \in T[j]} |L_k^T|)$, because each nonzero in i -th row does not necessarily apply to all ancestors due to the sparsity [10]. Therefore, a reasonable division of the supernodes will play a crucial role in the nonzeros amount and parallelism during the numerical factorization.

2.2 Relaxed Supernode and Factorization

In order to increase the granularity of the computing tasks, the size of the supernode is usually relaxed to merge node j and $j + 1$ together if their patterns are similar (not necessarily equal), which we called *relaxed supernodes*. At present, the most commonly used relaxation criteria are the additional number of padded zero entries, the sparsity and the column width of L factor. Generally, using this coarser partition in the factorization will introduce logically zero entries into the factor. For example, in Figure 1(b), (c), by treating the zero entries $l_{7,8}^T$ and $l_{8,9}^T$ as nonzeros, we can group $\{f, h, i\}$ as a relaxed supernode (purple dotted box).

After generating the relaxed supernodes, the algorithm will handle the numerical calculation. Based on the relaxed supernodal structure, the overall process will perform a series of dense Cholesky factorizations on each supernode. The nodes without ancestor-descendant relations have no data dependency, which means they can perform in parallel. As for those tasks with data dependencies, they must wait for all child nodes to finish before they can start computing. In (c) for instance, the root node f needs to wait supernodes $\{b, d\}$ and $\{c, e\}$ to complete their tasks, but $\{a, g\}$ can perform independently since it is a leaf node.

2.3 Challenges and Motivations

First of all, the division of relaxed supernodes directly affects the performance of sparse Cholesky factorization. We tested three common methods with the symmetric positive definite matrices from the SuiteSparse Collection [12], some results are shown in Table 1. It shows that the widely-used supernode merging strategy cannot adapt to different data sets, resulting in a fluctuation of nearly 30% in computing time. So a self-adapting merging strategy that can be aware of the matrix pattern will stabilize or even improve the performance of sparse Cholesky factorization.

Secondly, there is still much parallelism to be explored during the numerical factorization. Taking the current best-performing open-source library *SuiteSparse* as an example, the main contribution to the performance comes from the parallel implementation of BLAS kernels, and *OpenMP* for coarse-grained parallelism in assignment and reduction. Because of the size limitation of bottom leaf nodes, it is difficult to fully utilize the capacity of a large number of computational cores by only relying on dense BLAS parallelism, resulting in unsatisfactory performance. A proper task-level parallelization will improve the utilization of level-3 BLAS and provide finer control granularity between dense sub-tasks.

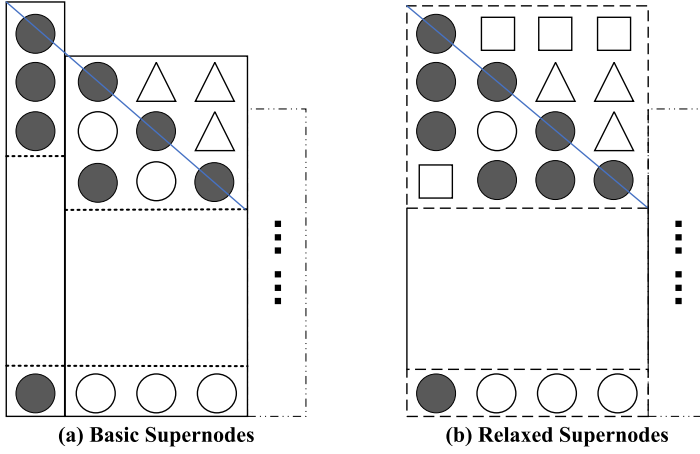


Fig. 2. An example of merging two fundamental supernodes S_i, S_{i+1} into a relaxed supernode $\hat{S}_i$.

Finally, as *multi-NUMA* (**Non-Uniform Memory Access**) is increasingly adopted in emerging architectures, the access speed gap between local and remote NUMA regions will also limit the performance of numerical operations. The supernodal Cholesky algorithm has a number of data updating operations between blocks. If the remote accesses are frequent during the factorization, it will seriously degrade the performance of the entire process. Currently, the core affinity is suitable for static scheduling, while task of supernodal Cholesky is irregular in nature, so it is necessary for us to design a new NUMA affinity scheme.

In the next sections, our contributions focus on these three typical problems respectively.

3 IMPROVE RELAXED SUPERNODES WITH NEURAL NETWORKS

3.1 Relaxation Parameters Analysis

Merging two supernodes in sparse matrix results in a reduction of sparse vector computations and data movement, but it comes at the expense of additional costs of dense matrix operations and increased storage requirements for logical zeros introduced into the factor L [2]. Figure 2 provides a simple example of merging two supernodes under the rule of [23], where the solid points indicate the original nonzeros and the circles represent fill-ins. The triangles and squares represent the extra zero elements introduced by the basic supernode generating process and the relaxed merging process, respectively.

Let S_i be the i -th fundamental supernode, $|S_i|$ denotes the number of nonzeros. Suppose S_i consists of N_i columns of vectors, and the corresponding lower triangular factor is denoted as L_{S_i} . The number of nonzero entries in column i is denoted by $NNZ(i)$. $R_k = \{S_{k,1}, \dots, S_{k,r}\}$ contains the row subscripts of r nonzeros of all columns of S_k . Take Figure 2 as an example, $R_1 = \{1, 2, 3, 8\}$ and $R_2 = \{2, 3, 4\}$. After merging into $\hat{S}_1$, the row subscripts set of $\hat{S}_1$ should be $\hat{R}_1 = R_1 \cup R_2 = \{1, 2, 3, 4, 8\}$. The maximum number of nonzeros of columns in S_k can be depicted as the width of R_k , which is recorded as $|R_k|$. So here $|\hat{R}_1|$ is equal to 5.

Suppose the factor L has a fundamental partition of basic supernodes $\mathcal{P}_B = \{S_1, S_2, \dots, S_K\}$, our goal is to find an optimal set of relaxed supernodes $\mathcal{P}_R = \{\hat{S}_1, \hat{S}_2, \dots, \hat{S}_M\}$ that minimizes the total *Cost* of inner factorization and update. For each supernode $\hat{S}$, it takes nearly $O(|\hat{S}|)$ time for inner factorization. A rank- k update is proportional to the time for k rank-1 update, which is also affected by the patterns of ancestors [8]. Due to the difficulties of accessing contribution blocks,

ALGORITHM 3.1: Finding the theoretical optimal solution for supernode merging

Input: The basic set of supernodes $\mathcal{P}_B = \{S_1, S_2, \dots, S_N\}$ which is postorder, the coefficient α, β , and cost of each S_i $Cost(S_i)$. A specific merging function $Merge(\hat{S}_{i-1}, S_i)$.

Output: The optimal relaxed supernodes partition $\mathcal{P}_R = \{\hat{S}_1, \hat{S}_2, \dots, \hat{S}_M\}$.

- 1: Initialize the total cost of subtrees: $C_t[1, \dots, N] = \{0\}$; Initialize relaxed partition: $\mathcal{P}_R = \mathcal{P}_B$.
- 2: $C_t[1] = Cost(S_1), C_t[2] = MIN\{C_t[1] + Cost(S_2), Merge(S_1, S_2)\}$;
- 3: **for** $i = 3$ to N **do**
- 4: $C_t[i] = MIN\{C_t[i-1] + Cost(S_i), C_t[i-2] + Merge(\hat{S}_{i-1}, S_i)\}$;
- 5: **if** $C_t[i] < C_t[i-1] + Cost(S_i)$ **then**
- 6: Modify the *Cost* of all reconstructed nodes $\{S_j, \dots, S_{i-1}; S_i\}$ in $\hat{S}_{i-1} \cup S_i$; // Merge occurs
- 7: **end if**
- 8: **end for**
- 9: Return $C_t[N]$ and $\mathcal{P}_R$ with M Relaxed Supernodes.

analyzing ancestor maps and evaluating floating-point operations [9], accurately quantifying the updating time is very challenging. In such cases, the updating time can be approximately estimated as $O(|L_{\hat{S}_i}|)$, where $|L_{\hat{S}_i}|$ refers to the number of nonzeros of the factor L . Therefore, we can use the following formula to approximately describe this problem:

$$\begin{aligned} \mathcal{P}_R = \min\{Cost_{fac} + Cost_{upt}\} &= \min \sum_{i=1}^M Cost(\hat{S}_i) = \min \sum_{i=1}^M [O(|\hat{S}_i|) + O(|L_{\hat{S}_i}|)] \\ &\approx \min\left\{\sum_{i=1}^M [\alpha \cdot |R_i| \cdot N_i + \beta \cdot D(i) \cdot NNZ(L_{\hat{S}_i})]\right\}, \end{aligned} \quad (4)$$

where M is the division result of relaxed supernodes, $D(i)$ is the depth of $\hat{S}_i$. The parameters α and β are used to approximate the cost of factorizing a dense matrix of size $|R_i| \times N_i$ and the cost of each rank- k update, respectively. The depth $D(i)$ provides a description of the complexity of updates. The coefficient α is related to the performance of dense BLAS/LAPACK operations, while β depends strongly on the matrix structure and communication between supernodes, such as memory bandwidth and NUMA locality, and the like. This formula is used to estimate the computational cost of the entire supernodal tree empirically. The relaxed supernode merging aims to minimize the cost of $\mathcal{P}_R$ by dynamically merging supernodes. It can be regarded as a dynamic programming problem that considers the parameters α , β , and merging rules. The theoretical solution to this problem can be obtained using Algorithm 3.1, which works from the bottom of the fundamental supernode tree.

However, finding the optimal solution is not a job that can be finished in polynomial time, because *Merge* function theoretically requires traversing all combinations of the subtrees in $\hat{S}_{i-1} \cup S_i$, which contains multiple basic nodes $\{S_j, \dots, S_{i-1}; S_i\} (O(2^{i-j}))$, to obtain the solution with the lowest cost. This leads to the fact that finding the theoretically optimal solution is not feasible in practice. In practical kernels (e.g., CHOLMOD, SuiteSparse), a greedy method is often applied to simplify the *Merge* rule to obtain a feasible solution. In addition, if we calculate *Cost* for all situations and keep C_t of all subtrees, this will introduce extra computational overhead and significant memory consumption.

To this end, many studies used some fixed thresholds as an alternative to the high-burden algorithm. In [2], Ashcraft and Grimes first took the number of padded zeros in $L_{\hat{S}_i}$ as the criterion, which used the *maxzero* as the threshold for supernodes merging. After experiments, they finally

ALGORITHM 3.2: Greedy strategy of merging supernodes by TRelax

Input: The basic set of supernodes $\mathcal{P}_B = \{S_1, S_2, \dots, S_N\}$. γ setting and $TRelax$ value of matrix A .

Output: A relaxed supernodes partition $\mathcal{P}_R = \{\hat{S}_1, \hat{S}_2, \dots, \hat{S}_M\}$.

- 1: **for** $i = 1$ to $N-1$ **do**
- 2: // Calculate computational granularity of S_i merged S_{i+1}
- 3: $T_{CG} = (|\hat{R}_{i,i+1}| \cdot N_{i,i+1} + \gamma \cdot Depth_i \cdot NNZ(i, i+1)) / Size_{\hat{L}_{i,i+1}}$;
- 4: **if** $T_{CG} \leq TRelax$ **then**
- 5: Nodes merging: $\hat{S}_i = S_i \cup S_{i+1}$, and adjust the size of reconstructed nodes.
- 6: **end if**
- 7: **end for**
- 8: Return $\mathcal{P}_R$ with M Relaxed Supernodes.

decided $maxzero = 128$ as a more general threshold for all matrices. In order to improve the adaptability, Chen et al. [8] considered numbers of columns ($nrelax$) and density of zeros introduced ($zrelax$) as relaxation parameters and set the threshold in stages. Although these two methods are currently widely-used, their limitations should not be ignored:

- They both lack fine-grained control of workload in supernodes (like *Cost* analyzed before) as the matrix block size and computing power fluctuates.
- They do not consider the 2D spatial structure of the matrix when determining a general strategy to fit the architecture, resulting in insufficient BLAS parallelism.

To achieve better performance in supernodal Cholesky, we must take computational cost into account, such that the effectiveness of supernodal load balance can be quantified and estimated. We must also exploit the pattern of sparse matrix to guide the merging process of the relaxed supernodes.

3.2 TRelax and Observations

For the first shortcoming, we design a novel relaxation parameter based on the computational granularity of each relaxed supernode $\hat{S}$, specifically:

$$TRelax = \frac{|\hat{R}| \cdot N + \gamma \cdot Depth \cdot NNZ}{Size_{\hat{L}}}, \quad (5)$$

where $Size_{\hat{L}}$ is the actual size of factor L of $\hat{S}$. In brief, $TRelax$ is derived from Equation (4) with two modifications. First, to reduce the uncertainty in evaluating α and β , we use $\gamma = \beta/\alpha$ to scale the *Cost* of the factorization and update parts, which simplifies estimation and improves usability. Second, $TRelax$ is normalized by $Size_{\hat{L}}$ and measures the relative granularity of workload. During the symbolic analysis, we control $TRelax$ of relaxed supernodes within specific ranges for different datasets, and use $TRelax$ as a dynamic criterion for merging. This parameter is considered more scalable and reasonable with an acceptable amount of additional computation while avoiding a large memory burden.

Algorithm 3.2 describes the practical merging strategy using TRelax, which greatly simplifies the complex *Cost* analysis in Algorithm 3.1. First we need to determine the values of γ for different matrices. Since the coefficient γ is the scaling factor of $Cost_{upt}$ relative to $Cost_{fac}$, we assume that γ should represent their ratio during the numerical phase. To support this claim, we made a simple observation in Figure 3(a), where the total factorizing and updating operations are denoted as

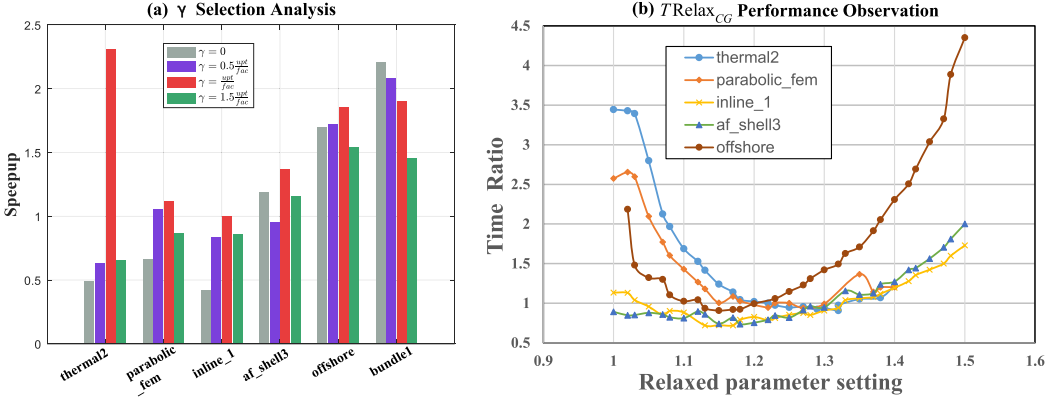


Fig. 3. (a) Speedups compared to maxNNZ method in different γ settings ($TRelax=1.2$). (b) Performance comparison of our new $TRelax$. The *Time Ratio* is calculated by the time with our new approach over the optimal time among those traditional strategies.

fac and upt , respectively. The experimental results confirm our hypothesis, showing that setting $\gamma = upt/fac$ leads to a superior speedup ratio.

Next, to evaluate the effectiveness of our new relaxation parameter, we conducted experiments on various ranges of $TRelax$ and made an observation, as shown in Figure 3(b). Each point represents a test routine where the computational granularity during the relaxed merging is controlled to not exceed the threshold $TRelax$. The results show that the performance of each dataset varies significantly with the variation of $TRelax$. In a certain range, it can outperform conventional partitions by nearly 20%, while in some intervals the performance becomes considerably worse. This observation validates the importance of selecting an appropriate $TRelax$ range to achieve optimal performance, as it controls the empirical computational granularity of each relaxed supernode. However, analytical modeling has difficulty in quantifying and capturing spatial features and sparse patterns, so we decide to use a well-trained deep learning model to adaptively infer a suitable range of $TRelax$ for each matrix, which will also address the second limitation.

3.3 Model Design

The primary challenge in this problem is determining the optimal range of $TRelax$ that adapts to different matrices, effectively controlling the granularity of supernodes. This requires a model that can perceive the sparse patterns of diverse matrices, informing the appropriate range of $TRelax$. Recently, the spatial-based **Graph Convolutional neural Network (GCN)** has been designed to exploit the structural information of graphs [26, 27, 44], and it is widely used in HPC [28, 29, 45, 46]. It captures the structural features of the corresponding graph (sparse matrix) via the message passing and features aggregation process between nodes during the training. Therefore, we decided to use the GCN model for adaptive adjustment of the $TRelax$ range.

Our 2-order GCN model is specifically designed for the extraction of sparse matrix patterns. It operates based on the principle of message propagation and aggregation, as illustrated in Figure 4. After abstracting the corresponding undirected graph G from matrix A , features are first aggregated on each vertex, then combined with the undirected graph G into a second-order graph for hidden structural feature extraction. This model, referred to as *High-order GCN* [34, 47], can be considered analogous to the merging process of relaxed supernodes, with the aim of learning relevant neighbor supernode properties during the training phase. In our approach, we associate a group of vertices with a specific fundamental supernode. These vertices undergo message propagation

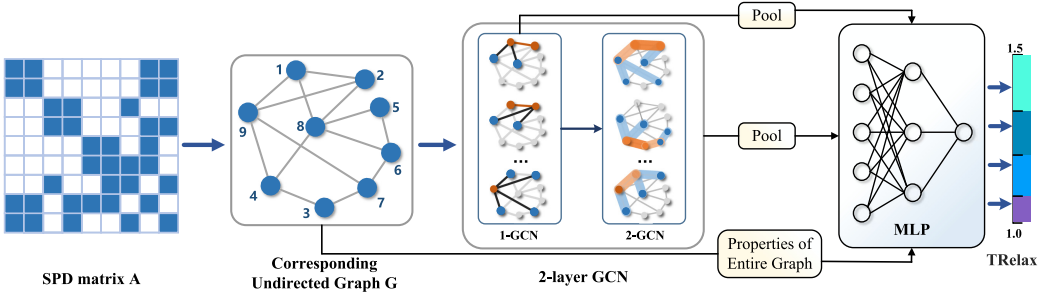


Fig. 4. Adaptive GCN model architecture for the optimal $TRelax$ range inference.

and aggregation processes with their respective neighborhoods, determined by the supernodes to which they belong, as shown in Figure 4. Suppose a vertex with depth- $(k-1)$ is annotated a feature $f_v^{(k-1)}$ in $\mathbb{R}^{1 \times D}$, the new feature of vertex v with depth- k is given by:

$$f_v^{(k)} = \sigma(f_v^{(k-1)} \cdot W_1^{(k)} + \sum_{n \in \mathcal{N}(v)} f_n^{(k-1)} \cdot W_2^{(k)}), \quad (6)$$

where $W_1^{(k)}, W_2^{(k)} \in \mathbb{R}^{D \times D'}$ are parameter matrices learned from training, $\mathcal{N}(v)$ is the set of neighbors excluded supernodal peers of vertex v , and $\sigma(\cdot)$ denotes a non-linear function where we use *ReLU*. After graph structure extraction, a *Top-K pooling* operation is done for each layer and additional properties of the whole matrix are supplemented before entering the classifier. Finally, a three-layer fully connected network is applied for the analysis of the above splicing features, and the optimal range of new relaxation parameter $TRelax$ is inferred by the *softmax* function.

3.4 Features and Ground Truth

To implement the aforementioned GCN model, it is essential to assign suitable features to the vertices and edges. Notably, the feature aggregation process takes place along the edges, making use of the inherent positional relationships of the nonzero elements (i.e., the edges of the graph). Furthermore, the values of nonzero elements do not affect the supernode partition, thus no additional features are set on the edges. In contrast, the vertices of graph G correspond to the rows/columns of matrix A , serving as the foundational units that constitute a supernode. As a result, the assignment of appropriate features to vertices can enhance the correlation and representativeness among rows and columns, leading to more precise inferences from the model.

Based on the previous analysis of the supernode merging process and Equation (4), we assign four features of the corresponding supernode to each vertex: (1) the depth of node; (2) the $|R|$ of node; (3) the column width; and (4) the size of lower triangular factor L . Note that these four are exactly the key variables in Equation (5) we derived. In addition, we supplement several extra properties in subsequent fully connected networks to enable our model to perceive the overall properties of the entire graph. The extra information we selected is shown below.

- **Size, NNZ and Sparsity** of matrix A ;
- $Flops/L_{nnz}$ calculated in symbolical analysis;
- **The average number of columns** of supernodes;
- **The depth and number of leaf nodes** of the fundamental supernodal tree.

The efficacy of our novel relaxation parameter $TRelax$ is directly mirrored in the numerical performance of Cholesky factorization. So we conducted supernodal Cholesky across a spectrum of $TRelax$ values and selected the configuration yielding the optimal performance as the ground

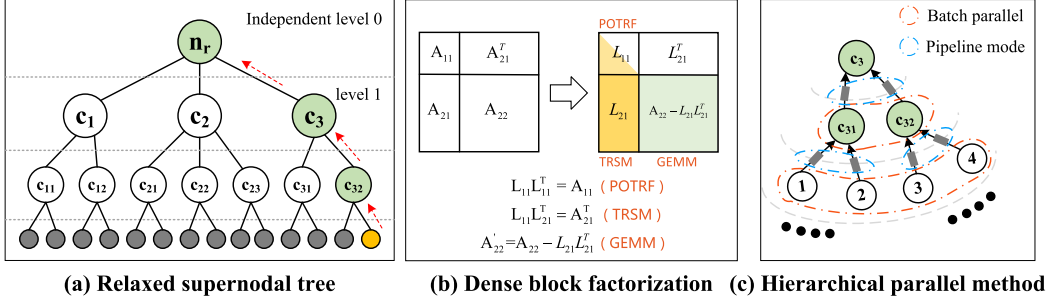


Fig. 5. (a) is an example of the relaxed supernodal tree. Each circle represents a *relaxed supernode* and each line means parent-child relationship. The gray dashed line divides the supernode tree into separate levels, and the red arrows show the direction of data assembly. (b) shows BLAS/LAPACK routines required in each sub-block factorization. The yellow part represents inner decomposition and the light green denotes assembling. (c) is our optimization strategy about hierarchical parallelism, where the gray squares represent the contributing blocks from the bottom.

truth for the matrix. In this work, we divided four types of datasets at $[1, 1.5]$ range based on the statistical distribution of 1,128 SPD matrices. The performance validation will select points of trisection from the inferred range and choose the one with optimal performance as the representation for its result. The GCN model can be viewed as a classification problem for different datasets to infer a suitable TRelax range. Our configuration of the model will be described in detail in the experimental section.

4 IMPROVING THE PERFORMANCE OF NUMERICAL FACTORIZATION

After the symbolical analysis, a sparse SPD matrix is divided into a string of supernodes to complete many dense factorizations, as shown in Figure 5(a). Each submatrix will perform the partial Cholesky factorization, as shown in Figure 5(b). The main dependent routines are **DPOTRF** (dense Cholesky factorization), **DTRSM** (triangular system solution), and **DGEMM/DSYRK** (general or symmetric matrix matrix multiplication) [37].

Therefore, the parallelization strategy of supernodal algorithm can be roughly divided into two modes: (1) one is that the task nodes are handled in serial, while the operations of each BLAS routine achieve in parallel [8]; (2) the other is to perform task-level parallelism between the supernodes [13, 14, 38]. Since the computing tasks are corresponding to supernodes, the granularity of parallelism in the first mode is relatively small for the bottom small-scale nodes resulting in a limited acceleration. For the second mode, due to the update operation changing the pattern of L , more synchronous operations are required, which affects the parallel efficiency. Therefore, with the increasing number of processor cores, it is necessary to combine two strategies to improve the parallelism of supernodal Cholesky factorization.

4.1 Hierarchical Parallel Optimization Method

In fact, the decomposition of a supernode can be functionally decoupled in two steps: first it integrates all contribution blocks from the descendants via GEMM/DSYRK, and then it performs inner factorization by POTRF and TRSM. Every time a node assembles the contribution blocks, the off-diagonal nonzeros in $L(:, n)$ will be updated. The specific updating algorithm we used is a modified version of *Algorithm.5* in [10]. Only when all of the child nodes have accomplished two steps, the parent can begin execution.

On top of this point, we designed a fine-grained **Hierarchical Parallel Supernodal (HPS)** Cholesky, which implements updating in the pipeline mode and partial factorizing in level

ALGORITHM 4.1: HPS Cholesky Numerical Factorization Algorithm

Input: The relaxed supernodal tree $\mathcal{T}_{relaxed}$ of A , which consists of task t_i .

```

1: for  $level = H$  to 1 do
2:   for nodes  $i$  in  $level$  do
3:     Determine which  $B_k$  will update  $t_i$ ;
4:     Arrange all contribution blocks in pipeline mode:
5:       Assemble supernodes (SYRK);
6:       Assemble lower panels (GEMM);
7:     Push updated  $t_i$  into local thread pool;
8:   end for
9:   Inner factorizing all tasks  $t_i$  in level-parallelism:
10:    Factor supernodes (POTRF);
11:    Factor lower panels (TRSM);
12:    Prepare its contribution block  $B_i$ ;
13:  end for

```

parallelism. A schematic diagram of our hierarchical parallel scheduling optimization is shown in Figure 5(c). Since the supernodal tree preserves the data dependencies, our strategy does not need to build a DAG graph like [6, 7]. Instead, we exploit the hierarchical structure of the tree to implement optimization. Here we take $\mathcal{T}(C_3)$ in Figure 5(c) as an example. Numerical factorization is performed from bottom to top. For levels that have multiple nodes, all nodes in each level (like node 1, 2, 3, 4 in $level.3$) are streaming scheduled to a thread pool so that they can be factorized immediately in parallel. Once a node finishes work and readies its contribution block, its parent node (like node C_{31}) will begin assembling operation. Note that the assembly is in a pipeline mode, which means all contribution blocks for a node are organized in an updating queue for modifying the nonzero pattern of their ancestor in turn.

In theory, the supernodes at the same level get the best opportunity for parallelism with each other, because they have similar task intensity and completely independent data structures. Consequently, parallelizing the inner factorization of the same-level nodes could gain significant benefits. Since the updating operation will modify the structure of columns of L , a subtree-based pipeline mode would be a better choice for its implementation. The number of children is used to synchronize the pipeline updating, and as a flag to start stream scheduling for subsequent inner factorization. Accordingly, the pipeline mode is a node synchronization policy for the next-level factorizing. In conclusion, HPS Cholesky provides a finer control granularity that arranges a series of thread-level factorizations and updates. The suggested implementation is presented as Algorithm 4.1.

4.2 Stream Scheduling on Multi-NUMA Device

In order to adapt to the multi-NUMA architecture, it is necessary to further optimize the task scheduling strategy for hierarchical parallelism. We set up an execution task queue on each NUMA node based on the parallelized strategy mentioned above. Specifically, when processing partial factorization of supernodes at the same level, sibling nodes are scheduled to the threads of the same NUMA region whenever possible. This can make the contribution blocks accessible locally during the pipeline updating process and avoid context switching costs by threads migration. Before the assembly, the parent node will be scheduled according to the location of most of its children, which minimizes data movement during updating. If a task cannot be assigned to the same NUMA region due to all local threads being busy or the task queue being full, it will be assigned nearby with reference to the *NUMA distance table* [41]. In addition, to avoid remote data fetching to improve

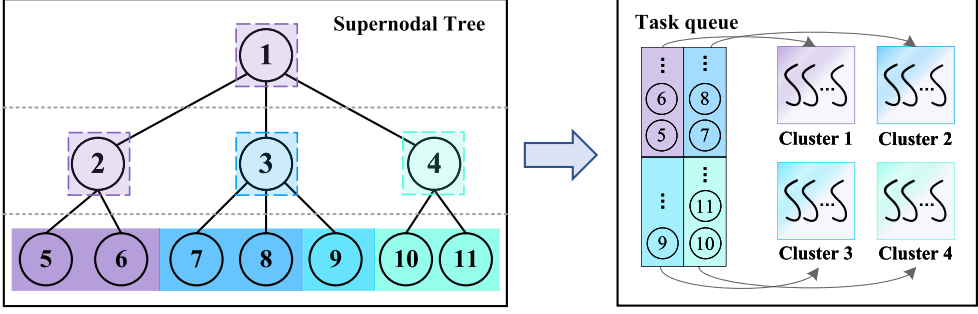


Fig. 6. Schematic diagram of multi-NUMA scheduling on a three-layer supernodal tree, where each color represents a NUMA region.

memory access efficiency, we strictly control the dense BLAS routines to perform on all available cores within the same NUMA node.

As shown in Figure 6, take a three-layer supernodal tree as an example, it will adaptively allocate supernodes according to the number of NUMA nodes. Supernodes on the same layer (*Node*5 ~ 11) will be arranged together by the scheduler to the appropriate thread queue and then stream scheduled to work threads. Each work thread will be bound to a NUMA region rather than to an individual core, allowing some leeway for local thread movement during the numerical factorization. *Node*2, 3 and 4 are assigned to the corresponding location where most of their children in, which allows them to get contribution blocks faster. Here we assume that up to two threads are created per NUMA node (with four different colors). The dense BLAS routines called by each task will be parallelized locally so that the data required by the dense routines can avoid remote access.

5 EXPERIMENTAL RESULTS

5.1 Experiments of New Relaxation Parameter

5.1.1 Implementation Details of Model. In this section, we carried out all GCN model experiments on *Nvidia A100-40G* with *Ubuntu 18.04 bionic* and *CUDA 11.2*. The *PyTorch Geometric package v2.0.4* is a necessary dependency library, which is implemented in *Python 3.8* and *PyTorch 1.11.0*. Similarly to [25], we adopt two layers of high-order GCN combined with three-layer MLP to balance computational cost and inference accuracy, based on our empirical study. The implementation details of our model for relaxation parameter are as follows:

- The first layer gets the input of a sparse graph and the dimension is (4,64). The second layer is expected to obtain deep structural attributes between nodes so we do not reduce the dimensionality. It is still (64,64).
- For each GCN layer, we record the explored results as x_1 , x_2 , and perform the maximum pooling operation $gmp(\cdot)$ separately. Then these features are catenated together with 7 properties of entire graph x_p . So we have $x_m = (gmp(x_1), gmp(x_2), x_p)$.
- Next, x_m is the input of multilayer perceptron with the structure of $(64 \times 2 + 7, 32, 16)$. After three levels of feature summarization, we can get a suitable range classification of *TRelax*.

There are 1,128 sparse matrices used in our work, of which 215 are symmetric positive definite matrices from the *University of Florida Sparse Matrix Collection* [12], and the rest are generated by *MATLAB* with random values. Since the adaptive classifier requires a rather large amount of GPU memory, we only used 693 datasets whose sizes are less than 100 thousand for the experiments. We divide them into a training set, validation set and test set by 8 : 1 : 1. The loss function we

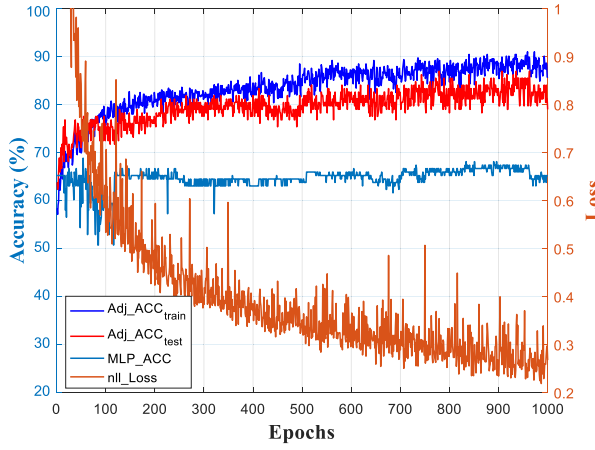


Fig. 7. The accuracy and loss results of our GCN model.

used is *Negative Log likelihood Loss*. Through experimental observations, we chose the following hyperparameters: $epochs = 1000$, $l_r = 0.0001$, $dropout = 0.2$. Note that calculating a large batch of data requires considerable GPU memory, so our $batch_size$ is set to 1. With this specified configuration, the complete training of all datasets requires approximately 6 hours. As the training is performed offline, prior to the sparse factorization, we consider this overhead to be acceptable.

5.1.2 Adaptive Classifier Results. The accuracy and loss results of our model are shown in Figure 7, where the brown line indicates the training error tends to converge. After 1,000 epochs of training, the GCN model with adjacent graph format can reach a maximum accuracy of 90.99% on the training set and 89.13% on the test set. We also tested two other data formats: bit graph [24] and hypergraph [35], which both are studied to improve the expressive power of aggregator. The results show that their test accuracy is 86.96% and 78.76% respectively. We speculate that this is because the data set used for Cholesky factorization is inherently symmetric positive definite, which leads to the corresponding adjacent graph obtaining sufficient decent aggregation attributes. If we only use multilayer perceptron to capture the entire graph properties, it can be seen from Figure 7 that the results are not very satisfactory (65.72%). In general, the MLP model is not accurate and generalized enough for the inference of relaxation parameter.

In conclusion, the range inference of our designed relaxation parameter T_{Relax} can be empowered by a well-trained GCN classifier model. Compared with the advanced aggregation algorithms, the simple adjacent graph format can achieve the highest accuracy up to 89.13%.

5.1.3 Performance Improvement by T_{Relax} . In order to evaluate the effect of our new relaxation parameter in supernodal Cholesky, we collected the performance results of all stages for eight SPD matrices from SuiteSparse Collection. The results are shown in Figure 8, where the shaded area is the overhead of symbolic analysis, and the rest represents the numerical part.

The 'Maxzero' method only relies on the padded logical zeros in relaxed supernodes, thus having the fastest symbolical factorization. For the 'Progressive' method, which is the approach of progressive judgment using the column width and sparsity of relaxed supernodes, its symbolical analysis is less expensive and has fairish numerical performance. As for our parameter T_{Relax} , the analysis part (red bar) is relatively large due to the additional computational overhead. However, the performance of the numerical part is significantly improved because of its fine-grained control of the supernodal computation. In addition, the inference time of GCN model is negligible,

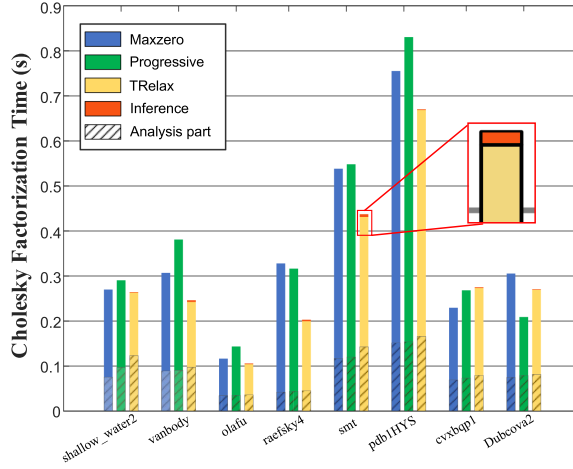


Fig. 8. Time cost under different relaxed supernodes merging methods. 'Progressive' denotes a traditional method which uses *nnz* and *columnwidth* to progressively perform merge judgment. 'Maxzero' indicates a strategy that only relies on the number of padded zeros. 'TRelax' is the parameter proposed in this paper, including the inference time.

typically it takes about 10^{-3} seconds. Despite the fact that the model requires approximately 0.5 seconds to warm up, we are confident that this additional overhead can be amortized effectively when computing multiple matrices. For the last two datasets without significant improvement, we deduce that the range of *TRelax* needs to be modified exclusively since the uniform threshold range may be not applicable enough for their distinguished blocks.

Overall, our new relaxation parameter *TRelax* can outperform the 'Progressive' and 'Maxzero' methods on nearly 66.15% and 71.22% of the 693 datasets, and delivers up to 1.25x and 1.33x speedups separately.

5.2 Improvement of Numerical Parallelization

In this section, we perform our optimization to the numerical phase of supernodal factorization on three platforms: *Huawei Taishan Server*, with 2 Kunpeng 920 processors, 4 NUMA nodes; *Intel Xeon Gold 6248* with 1 processor, 1 NUMA node and *Phytium 2000+* with 1 processor, 8 NUMA nodes. We evaluate the performance improvement of the 8 sparse matrices in Table 2 on each platform. These representative datasets, which arise from Fluid Dynamics, Structural and PDE solver problems, are also used as *TRelax* cases in Figure 8.

When performing supernodal Cholesky factorization on these platforms, we notice that for most matrices, the best performance is delivered when the number of multi-thread of dense BLAS is set to 32, 20, and 8, respectively. This is because they are the number of cores within a single NUMA node for each platform. We carried out experiments of the SuiteSparse dynamic supernodal Cholesky, PaStiX Cholesky and our HPS Cholesky. PaStiX uses the default SCOTCH reordering algorithm, while SuiteSparse and HPS both use the **approximate minimum degree (AMD)** reordering algorithm. The dense operations for all three sparse Cholesky rely on OpenBLAS 0.3.23, the numerical performance comparisons are shown in Figure 9.

As shown in Figure 9, our HPS Cholesky achieves different performance improvements on these systems. Specifically, on Intel Xeon 6248 system, our implementation's performance advantage is not significant enough, which is roughly on-par with SuiteSparse(0.97x) and PaStiX(1.07x). Fortunately, our parallel optimization strategy delivers satisfactory speedup on multi-NUMA systems.

Table 2. Information of the Eight Representative Matrices

Matrix	Size	nnz(A)	Sparsity	Flops/Lnz
shallow_water2	81920	327680	4.88E-05	248.01
vanbody	47072	2336898	1.05E-03	261.70
olafu	16146	1015156	3.89E-03	365.23
raefsky4	19779	1328611	3.40E-03	632.73
smt	25710	3753184	5.68E-03	882.71
pdb1HYS	36417	4344765	3.28E-03	990.13
cvxbqp1	50000	349968	1.40E-04	794.66
Dubcova2	65025	1030225	2.44E-04	160.16

The *Flops/Lnz* is the ratio of floating points operations of Cholesky factorization to the number of nonzeros in *L*.

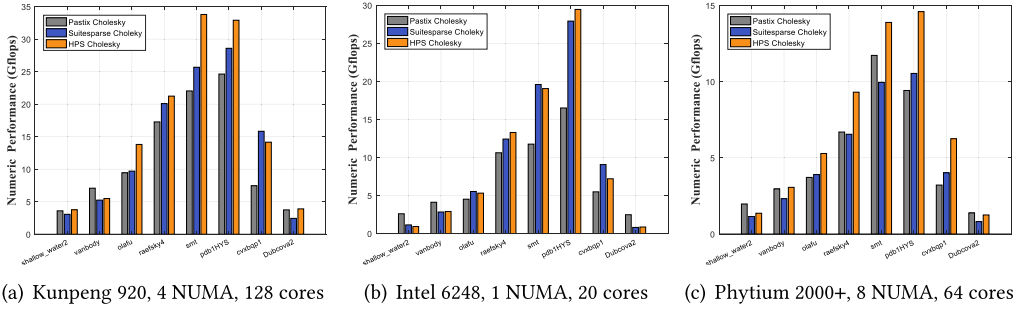


Fig. 9. Performance comparison of eight SPD matrices on Kunpeng 920, Intel 6248 and Phytium 2000+.

On Kunpeng 920, the inner partial factorization can gain significant performance benefits over SuiteSparse(1.21x) and PaStiX(1.29x) because of the hierarchical parallelism and NUMA-locality scheduling optimization. Similarly, on Phytium 2000+ with 8 NUMA nodes, HPS Cholesky can achieve higher average speedup ratios of 1.39x and 1.27x compared to SuiteSparse and PaStiX, respectively.

From the results, the hierarchical parallel algorithm and the NUMA affinity scheduling method have satisfactory performance advantages on multi-NUMA architecture platforms. The expected performance improvement in a single NUMA system is not sufficient to offset the extra overheads due to the additional introduction of data pre-placement and a more detailed thread-queue management mechanism. Note that PaStiX is able to reach relatively high performance on those datasets of low computational intensity (*shallow_water2*, *vanbody* and *Dubcova2*), which is achieved through a more time-consuming symbolic analysis process, as elaborated in Section 5.4.

5.3 Roofline Model Assessment

The roofline model is a visual model used to give a performance estimate of a specific kernel or application [42]. It proposes a quantitative analysis method using operation intensity, and presents a formula about the upper limit of theoretical peak performance on the computing platform. In this section, we perform a detailed roofline model evaluation of the three libraries tested before on a multi-NUMA system (Kunpeng 920), and then quantify our optimization strategies based on the assessment.

First, we ran the memory bandwidth benchmark and the results are shown in Figure 10. All memory pages and the threads always reside in the same NUMA domain via the *numactl* API. Referring to the data access during the supernodal Cholesky, multi-threaded read-only and

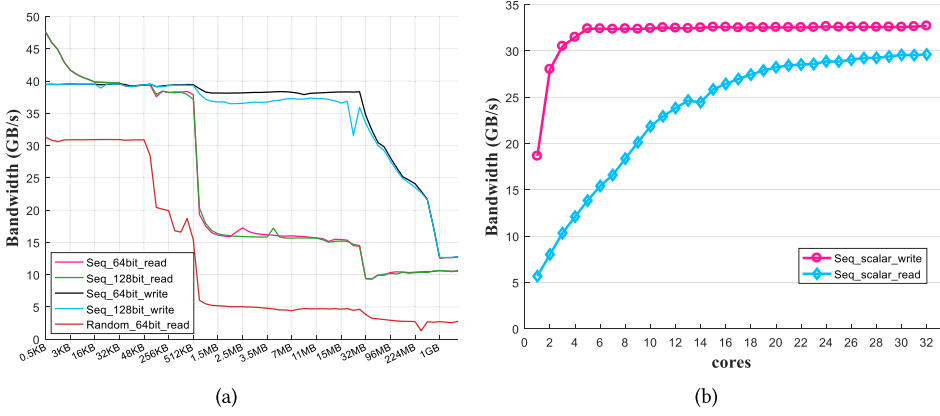


Fig. 10. DRAM bandwidth test inside a NUMA region on Kunpeng 920. (a) The bandwidth of varying sizes within a single thread. (b) The bandwidth of 10GB block with varying threads in a NUMA domain.

write-only bandwidths are recorded in scalar, double precision. Figure 10(b) shows if enough cores are used then read-only and write-only benchmarks saturate the memory bandwidth.

Then we can obtain the theoretical double precision peak floating-point performance of Kunpeng 920 by $128 \text{ cores} \times 2.6 \text{ GHz} \times 128 \text{ bit vector} / 64 \text{ bit} * 2 \text{ FLOP} = 1331.2 \text{ Gflops/s}$. However, almost all sparse kernels and applications are not expected to reach the maximum performance. This is due to memory intensive nature that reduces the operation intensity, resulting in the code's performance being limited by memory bandwidth, which is called memory-bound.

Here we adopt a *Modified Roofline Model* [43], whose target is sparse triangular solve via sparse Cholesky or LDL^T factorization, to assess the implementation. The attainable memory-bound performance can be predicted by the formula:

$$p^A(t) = \frac{nnz(L) \times \frac{FLOP}{nz} \frac{FLOP}{s}}{D^A(t)/B(t)}, \quad (7)$$

where $nnz(L)$ denotes the total number of nonzeros of all factor L during the factorization of matrix A for t threads, $D^A(t)$ is the data and indices volume that make up L , and $B(t)$ is the attainable memory bandwidth by t threads. Since the factorization of each supernode is assigned to threads inside a NUMA domain to exploit the BLAS parallelism, we set $t = 32$ to perform the evaluation. So based on formula 7 driven in the paper [43], we assessed the implementation of SuiteSparse, PaStiX and HPS Cholesky, the summary results of the eight sparse matrices are shown in Figure 11.

For the purpose of presenting a detailed roofline analysis on each dataset, we partitioned the datasets into three distinct subfigures, denoted as (a), (b), and (c), based on their operation intensities. Generally, our HPS exhibits an improved operation intensity for most datasets, leading to a substantial enhancement in the theoretically attainable performance. This improvement can be primarily attributed to the adaptive merging strategy based on the relaxation parameter that enables each relaxed supernode to achieve better two-level parallelism. For datasets with relatively high operation intensity (*oalfu,raefsky,cvxbq1, etc.*), HPS Cholesky also obtains a larger performance gain compared to the slope of roofline model, which is the contribution of hierarchical parallel optimization. On the *vanbody* dataset, PaStiX operates with much higher computational intensity and reaches higher performance than the other two implementations. This is mainly due to the different matrix structures formed by sophisticated block repartitioning and scheduling phases [22], which is beyond the scope of this paper.

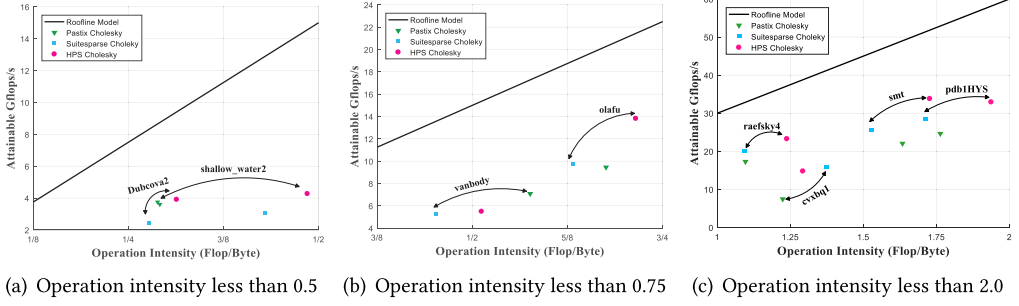


Fig. 11. Comparison results (double precision) of roofline model analysis about eight sparse matrices on Kunpeng 920.

In conclusion, our HPS Cholesky demonstrates superior performance in memory-bound sparse computations. Specifically, HPS is capable of extracting on average nearly 49.1% of the Kunpeng 920's theoretical peak bandwidth, as compared to 43.3% achieved by SuiteSparse and 44.5% by PaStiX. For the other platforms, the bandwidth utilization has similar results, although there is a difference in the significance of the acceleration effect of hierarchical parallelization due to the NUMA phenomenon. Therefore, the effectiveness of our optimization approaches in achieving high-performance supernodal Cholesky on a multi-NUMA system is demonstrated by the roofline analysis results. Particularly for datasets with multiple leaves, such as *smt* and *pdb1HYS*, HPS achieves satisfactory Gflops while also improving the operation intensity.

5.4 Performance Comparison with Existing Work

5.4.1 Multi-platform Performance Comparison. In order to compare the overall performance over existing open-source libraries, all optimization strategies are applied to relaxed supernodal Cholesky, including new relaxed parameter *TRelax* and hierarchical parallelization. Note that NUMA affinity and streaming scheduling policies are already implemented by default at the thread pool framework.

Figure 12 shows the performance comparison of running double precision sparse Cholesky factorization on the 215 SPD sparse matrices with Simplicial Cholesky of Eigen 3.3.9, Simplicial LL^T and Supernodal LL^T from CHOLMOD 5.2.0, adaptive supernodal of SuiteSparse 5.7.2, PaStiX Cholesky 6.2.2 and our work HPS Cholesky on Taishan Server. What's more, on Xeon Gold 6248 as shown in Figure 12(b), we also run *Intel MKL PARDISO Cholesky* [40], which can be regarded as a gold standard for the highest performance of sparse Cholesky factorization. To calculate the runtime performance, we consider the entire factorization process, including both symbolic analysis and numerical factorization. Because for each dataset, these steps are indispensable and should be included in the calculation of overall performance.

The six subfigures of Figure 12(a) and (b) illustrate that simplicial Cholesky factorization provides very low performance due to insufficient computation granularity between elimination nodes. The native implementation of *Eigen's* interface exhibits the worst performance among sparse Cholesky methods, and the result points are almost uniformly distributed along the linear regression line. Similarly, *CHOLMOD Simplicial LL^T* achieves comparable results, but only a small portion of the datasets show performance close to 5 Gflops. *CHOLMOD supernodal LL^T* algorithm provides high-performance supernodal Cholesky, but it lacks the ability to adapt to different matrix sizes. In contrast, *SuiteSparse* dynamically employs the relaxed supernodal method based on *Flops/lnz*, resulting in excellent overall performance. In previous sections, experiments conducted on *PaStiX* demonstrate its outstanding numerical performance and bandwidth utilization.

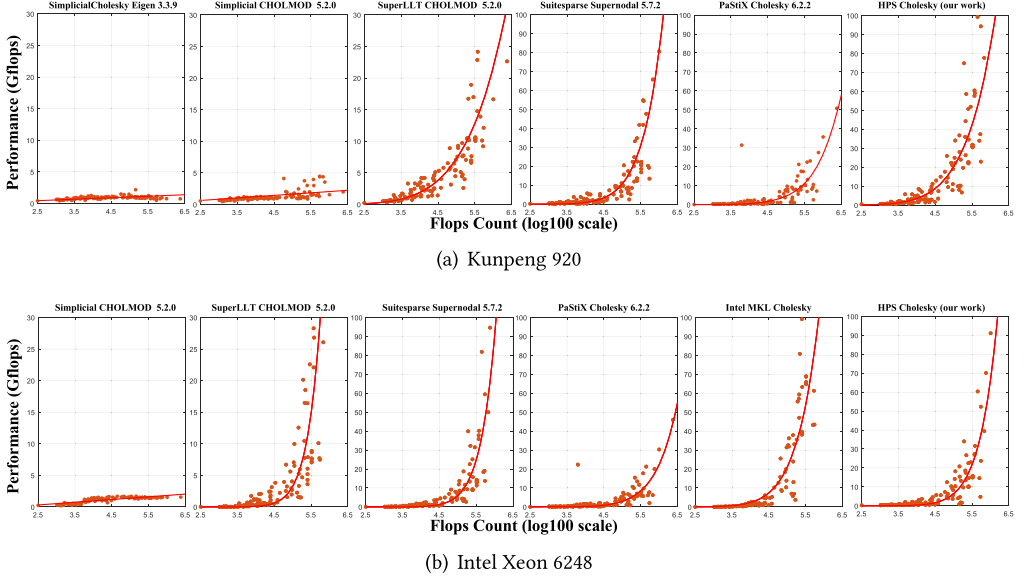


Fig. 12. Performance comparison of HPS Cholesky and other state-of-the-art libraries on (a) Kunpeng 920 and (b) Intel Xeon 6248. The six subfigures show double precision performance (GFlops) and their linear/power regression of 215 SPD sparse matrices from SuiteSparse Collection. To better show the comparison effect, the x-axis is the theoretical floating points amount in $\log_{100}$ scale.

However, its runtime performance appears to be less competitive when the time required for symbolic analysis is taken into account. This is because it employs block repartitioning and static scheduling analysis during the symbolic phase, which helps to balance memory consumption and performance but incurs extra cost. These features make it an excellent option for analyzing and solving a series of matrices with similar structures.

In general, our implementation of HPS Cholesky achieves comparable performance to SuiteSparse and PaStiX at small computational scales on both Kunpeng and Intel platforms. Fortunately, when the computational amount surpasses 10^9 ($x \geq 4.5$), HPS Cholesky provides a noticeable performance improvement over SuiteSparse. The results show that, when comparing the data performance on Kunpeng and Intel platforms, 92.09% and 75.18% respectively outperform SuiteSparse, while 93.02% and 90.22% respectively outperform PaStiX. The performance curve of HPS Cholesky on Intel Xeon 6248 is similar to that on KunPeng but the advantages are not as significant as on the 4-NUMA Kunpeng platform. In addition, HPS achieves satisfactory numerical performance with a faster symbolic analysis process compared to PaStiX. These findings indicate that our HPS is well-suited for both platforms and can achieve more competitive performance on multi-NUMA systems.

Figure 13 depicts the scatter plots of the performance ratio of HPS Cholesky versus MKL sparse Cholesky on SuiteSparse datasets, where the points above the red line indicate better performance of our method. Our analysis reveals that HPS outperforms MKL at dimensions below 15,000. While MKL PARDISO Cholesky has a significant performance advantage for large-scale matrices ($> 100K$), our HPS Cholesky surpasses the current optimal standard on small to medium-sized datasets.

5.4.2 Statistical Results of 1,128 Datasets. Furthermore, to verify the robustness of the optimization, next we did an overall performance test on all 1,128 sparse matrices. The analysis in this

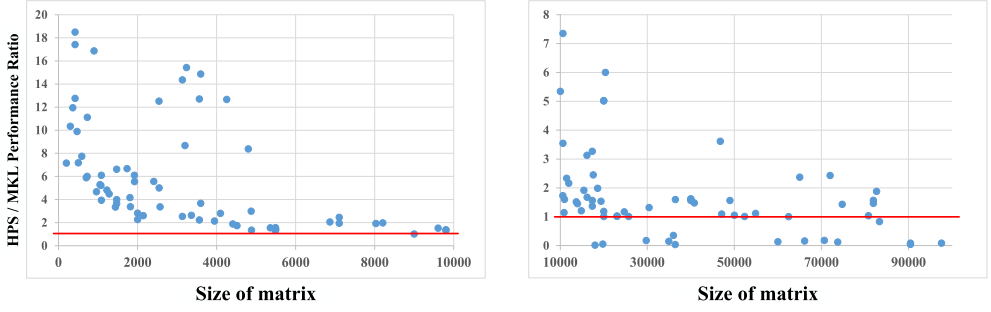


Fig. 13. Performance comparison of HPS Cholesky and MKL on 215 SuiteSparse collection.

Table 3. Statistical Results of HPSCholesky on Kunpeng 920

Compared with 215 SuiteSparse datasets		
	speedup > 1.0x	The proportion
Eigen- LL^T	200	93.02%
CholmodSimplicial	186	86.51%
CholmodSuper LL^T	201	93.49%
PaStiX	200	93.02%
SuiteSparse	198	92.09%
Intel MKL PARDISO	171	79.53%
Compared with all 1,128 datasets		
	speedup > 1.0x	The proportion
Eigen- LL^T	900	79.78%
CholmodSimplicial	898	79.60%
CholmodSuper LL^T	1015	89.98%
PaStiX	926	82.09%
SuiteSparse	840	74.47%
Intel MKL PARDISO	799	70.83%

section is mainly carried out for the 4-NUMA Kunpeng 920. The complementary dataset is our randomly generated sparse SPD matrices whose sizes are less than 20,000 and densities not higher than 5%. Table 3 shows the statistical results of performance on all data sets.

It shows that almost 90% of SPD matrices from SuiteSparse Collection have been accelerated compared to the state-of-the-art libraries on multi-NUMA systems, which means it gains promising performance benefits in real-world engineering and structural problems. After supplementing the tests with extra 913 data sets we randomly generated, the performance results are shown in the lower part of Table 3. In general, our HPSCholesky still runs faster than other six routines on 900, 898, 1015, 926, 840 and 799 matrices, with an average speedup of 35.63x, 32.89x, 4.28x, 3.22x, 1.41x and 2.42x (up to 136.00x, 98.44x, 9.38x, 11.73x, 10.31x and 92.32x), respectively. The maximum performance drops are 14.69%, 23.19% and 36.29% compared to SuiteSparse, PaStiX and MKL separately. The achieved performances are rather good and confirm that our adaptive relaxed parameter and hierarchical parallel strategy are suitable for multicore and multi-NUMA architectures. Especially for datasets with large *FLOPs* (often with a large number of supernodes), our relaxed supernodal Cholesky implementation is more competitive compared to the current vendor-supplied sparse libraries. Compared with Intel MKL, our HPSCholesky still achieves considerable performance advantages on medium-sized datasets.

6 RELATED WORK

Efficient solution of sparse systems is facilitated by two widely used algorithms: the supernodal algorithm [8, 13] and the multifrontal algorithm [18–20]. Both these algorithms are based on an elimination tree and perform matrix operations on dense blocks, leveraging level-3 BLAS kernels. Compared with supernodes, multifrontal methods need a particular scheduling strategy and tend to use more storage space [21]. Liu summarized the detailed principle and implementation of multifrontal algorithms in different computational environments, as articulated in [32].

As for the supernodal algorithm, Ashcraft and Grimes [2] first described diverse enhancements of supernodes merging and evaluated a new implementation to reduce the number of sparse computations. Furthermore, [2, 30] introduced the *Relaxed Supernodes*, which is an extension of the basic supernodal strategy by merging two similar supernodes in factor L . The package *PAR-DISO* [39, 40] carried out a pipeline mode of parallelism for the supernodal factorization. Then *PaStiX* [22, 36] studied the scalability and focused on block partitioning and scheduling to balance memory overhead and efficiency. On top of [9, 10], Chen et al. [8] further provided a highly optimized implementation *CHOLMOD* for sparse supernodal Cholesky. Then Davis and Hager [11] described a dynamic supernodes algorithm to obtain better performance in *SuiteSparse*. At present it has become one of the most commonly used open source libraries for sparse Cholesky factorization. This paper focuses on the twofold optimizations of the symbolic analysis and numerical factorization based on the supernodal algorithm. Differently from those methods, our HPS Cholesky optimizes the supernode merging process with an adaptive parameter $TRelax$ empowered by GCN and focuses on hierarchical parallelization with NUMA affinity.

7 CONCLUSION

In this work, we propose HPS Cholesky, a hierarchical parallel optimization of sparse supernodal Cholesky factorization, with a novel relaxation parameter $TRelax$ empowered by GCN. Our method focuses on both symbolical analysis and numerical factorization, as reflected by fine grained computational control over relaxed supernodes, great task-level parallelism, and a pervasive NUMA-affinity scheduling implementation. Based on the experimental results, it can be concluded that the performance of HPS Cholesky is comparable to that of SuiteSparse and PaStiX on single NUMA systems. Moreover, our optimization method can achieve promising performance improvement on multi-NUMA systems, particularly for small and medium-sized sparse matrices. The implementation of HPS Cholesky in this paper has been released to <https://github.com/lsl036/HPS-Cholesky>.

REFERENCES

- [1] Saeid Abbasbandy, Ahmad Jafarian, and Reza Ezzati. 2005. Conjugate gradient method for fuzzy symmetric positive definite system of linear equations. *Applied Mathematics and Computation* 171, 2 (2005), 1184–1191.
- [2] Cleve Ashcraft and Roger Grimes. 1989. The influence of relaxed supernode partitions on the multifrontal method. *ACM Transactions on Mathematical Software (TOMS)* 15, 4 (1989), 291–309.
- [3] Cleve Ashcraft and Roger G. Grimes. 1999. SPOLES: An object-oriented sparse matrix library. In *PPSC*.
- [4] N. Bitoulas and M. Papadrakakis. 1994. An optimized computer implementation of incomplete Cholesky factorization. *Computing Systems in Engineering* 5, 3 (1994), 265–274.
- [5] Stephen Cauley, Venkataramanan Balakrishnan, and Cheng-Kok Koh. 2010. A parallel direct solver for the simulation of large-scale power/ground networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 29, 4 (2010), 636–641.
- [6] Xiaoming Chen, Wei Wu, Yu Wang, Hao Yu, and Huazhong Yang. 2011. An escheduler-based data dependence analysis and task scheduling for parallel circuit simulation. *IEEE Transactions on Circuits and Systems II: Express Briefs* 58, 10 (2011), 702–706.
- [7] Xiaoming Chen, Lixue Xia, Yu Wang, and Huazhong Yang. 2016. Sparsity-oriented sparse solver design for circuit simulation. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1580–1585.

- [8] Yanqing Chen, Timothy A. Davis, William W. Hager, and Sivasankaran Rajamanickam. 2008. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software (TOMS)* 35, 3 (2008), 1–14.
- [9] Timothy A. Davis and William W. Hager. 1999. Modifying a sparse Cholesky factorization. *SIAM J. Matrix Anal. Appl.* 20, 3 (1999), 606–627.
- [10] Timothy A. Davis and William W. Hager. 2001. Multiple-rank modifications of a sparse Cholesky factorization. *SIAM J. Matrix Anal. Appl.* 22, 4 (2001), 997–1013.
- [11] Timothy A. Davis and William W. Hager. 2009. Dynamic supernodes in sparse Cholesky update/downdate and triangular solves. *ACM Transactions on Mathematical Software (TOMS)* 35, 4 (2009), 1–23.
- [12] Timothy A. Davis and Yifan Hu. 2011. The University of Florida Sparse Matrix Collection. *ACM Transactions on Mathematical Software (TOMS)* 38, 1 (2011), 1–25.
- [13] James W. Demmel, Stanley C. Eisenstat, John R. Gilbert, Xiaoye S. Li, and Joseph W. H. Liu. 1999. A supernodal approach to sparse partial pivoting. *SIAM J. Matrix Anal. Appl.* 20, 3 (1999), 720–755.
- [14] James W. Demmel, John R. Gilbert, and Xiaoye S. Li. 1999. An asynchronous parallel supernodal algorithm for sparse Gaussian elimination. *SIAM J. Matrix Anal. Appl.* 20, 4 (1999), 915–952.
- [15] Florin Dobrian, Gary Kumbfert, and Alex Pothen. 2000. The design of sparse direct solvers using object-oriented techniques. In *Advances in Software Tools for Scientific Computing*. Springer, 89–131.
- [16] Jack J. Dongarra, Jerney Du Cruz, Sven Hammarling, and Iain S. Duff. 1990. Algorithm 679: A set of level 3 basic linear algebra subprograms: Model implementation and test programs. *ACM Transactions on Mathematical Software (TOMS)* 16, 1 (1990), 18–28.
- [17] Jack J. Dongarra, Jeremy Du Croz, Sven Hammarling, and Iain S. Duff. 1990. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software (TOMS)* 16, 1 (1990), 1–17.
- [18] Iain S. Duff and John K. Reid. 1983. The multifrontal solution of indefinite sparse symmetric linear. *ACM Transactions on Mathematical Software (TOMS)* 9, 3 (1983), 302–325.
- [19] Iain S. Duff and John K. Reid. 1984. The multifrontal solution of unsymmetric sets of linear equations. *SIAM J. Sci. Statist. Comput.* 5, 3 (1984), 633–641.
- [20] Stanley C. Eisenstat, Martin H. Schultz, and Andrew H. Sherman. 1978. *Software for Sparse Gaussian Elimination with Limited Core Storage*. Technical Report. Yale Univ. New Haven Conn. Dept. of Computer Science.
- [21] Nicholas I. M. Gould, Jennifer A. Scott, and Yifan Hu. 2007. A numerical evaluation of sparse direct solvers for the solution of large sparse symmetric linear systems of equations. *ACM Transactions on Mathematical Software (TOMS)* 33, 2 (2007), 10–es.
- [22] Pascal Hénon, Pierre Ramet, and Jean Roman. 2002. PaStiX: A high-performance parallel direct solver for sparse symmetric positive definite systems. *Parallel Comput.* 28, 2 (2002), 301–321.
- [23] Jonathan D. Hogg, John K. Reid, and Jennifer A. Scott. 2010. Design of a multicore sparse Cholesky factorization using DAGs. *SIAM Journal on Scientific Computing* 32, 6 (2010), 3627–3649.
- [24] Yongcheng Jing, Yiding Yang, Xinchao Wang, Mingli Song, and Dacheng Tao. 2021. Meta-aggregator: Learning to aggregate for 1-bit graph neural networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 5301–5310.
- [25] Thomas N. Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [26] Liang Li, Siwei Wang, Xinwang Liu, En Zhu, Li Shen, Kenli Li, and Keqin Li. 2022. Local sample-weighted multiple kernel clustering with consensus discriminative graph. *IEEE Trans. Neural Networks* (2022), 1–14. <https://doi.org/10.1109/TNNLS.2022.3184970>
- [27] Liang Li, Junpu Zhang, Siwei Wang, Xinwang Liu, Kenli Li, and Keqin Li. 2023. Multi-view bipartite graph clustering with coupled noisy feature filter. *IEEE Trans. Knowl. Data Eng.* (2023), 1–13. <https://doi.org/10.1109/TKDE.2023.3268215>
- [28] Shengle Lin, Wangdong Yang, Haotian Wang, Qinyun Tsai, and Kenli Li. 2021. STM-multifrontal QR: Streaming task mapping multifrontal QR factorization empowered by GCN. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–14.
- [29] Jiaxiang Liu, Jin Hao, Hangzheng Lin, Wei Pan, Jianfei Yang, Yang Feng, Gaoang Wang, Jin Li, Zuolin Jin, Zhihe Zhao, and Zuozhu Liu. 2023. Deep learning-enabled 3D multimodal fusion of cone-beam CT and intraoral mesh scans for clinically applicable tooth-bone reconstruction. *Patterns* 4, 9 (2023).
- [30] Joseph W. H. Liu. 1990. The role of elimination trees in sparse factorization. *SIAM Journal on Matrix Analysis and Applications* 11, 1 (1990), 134–172.
- [31] Joseph W. H. Liu. 1991. A generalized envelope method for sparse factorization by rows. *ACM Transactions on Mathematical Software (TOMS)* 17, 1 (1991), 112–129.

- [32] Joseph W. H. Liu. 1992. The multifrontal method for sparse matrix solution: Theory and practice. *SIAM Review* 34, 1 (1992), 82–109.
- [33] Joseph W. H. Liu, Esmond G. Ng, and Barry W. Peyton. 1993. On finding supernodes for sparse matrix computations. *SIAM J. Matrix Anal. Appl.* 14, 1 (1993), 242–252.
- [34] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. 2019. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4602–4609.
- [35] Eliya Nachmani and Lior Wolf. 2019. Hyper-graph-network decoders for block codes. *Advances in Neural Information Processing Systems* 32 (2019).
- [36] Grégoire Pichon, Eric Darve, Mathieu Faverge, Pierre Ramet, and Jean Roman. 2018. Sparse supernodal solver using block low-rank compression: Design, performance and analysis. *Journal of Computational Science* 27 (2018), 255–270.
- [37] Steven C. Rennich, Darko Stosic, and Timothy A. Davis. 2016. Accelerating sparse Cholesky factorization on GPUs. *Parallel Comput.* 59 (2016), 140–150.
- [38] Edward Rothberg. 1996. Performance of panel and block approaches to sparse Cholesky factorization on the iPSC/860 and Paragon multicomputers. *SIAM Journal on Scientific Computing* 17, 3 (1996), 699–713.
- [39] Olaf Schenk and Klaus Gärtner. 2002. Two-level dynamic scheduling in PARDISO: Improved scalability on shared memory multiprocessing systems. *Parallel Comput.* 28, 2 (2002), 187–197.
- [40] Olaf Schenk, Klaus Gärtner, Wolfgang Fichtner, and Andreas Stricker. 2001. PARDISO: A high-performance serial and parallel sparse linear solver in semiconductor device simulation. *Future Generation Computer Systems* 18, 1 (2001), 69–78.
- [41] Sean Williams, Latchesar Ionkov, and Michael Lang. 2017. NUMA distance for heterogeneous memory. In *Proceedings of the Workshop on Memory Centric Programming for HPC*. 30–34.
- [42] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
- [43] Markus Wittmann, Georg Hager, Radim Janalik, Martin Lanser, Axel Klawonn, Oliver Rheinbach, Olaf Schenk, and Gerhard Wellein. 2018. Multicore performance engineering of sparse triangular solves using a modified roofline model. In *2018 30th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*. IEEE, 233–241.
- [44] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S. Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 32, 1 (2020), 4–24.
- [45] Yong Yang, Jiaxiang Liu, Shuying Huang, Weiguo Wan, Wenying Wen, and Juwei Guan. 2021. Infrared and visible image fusion via texture conditional generative adversarial network. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 12 (2021), 4771–4783. <https://doi.org/10.1109/TCSVT.2021.3054584>
- [46] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng, and Haifeng Li. 2019. T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE Transactions on Intelligent Transportation Systems* 21, 9 (2019), 3848–3858.
- [47] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81.
- [48] Shuisheng Zhou. 2015. Sparse LSSVM in primal using Cholesky factorization for large-scale problems. *IEEE Transactions on Neural Networks and Learning Systems* 27, 4 (2015), 783–795.

Received 25 November 2022; revised 4 September 2023; accepted 16 October 2023